1. Mục tiêu

- Hiểu rõ sự khác biệt giữa **Monolithic**, **Modular Monolithic** và **Microservices** — không chỉ về mặt định nghĩa mà quan trọng hơn là **khi nào nên chọn kiến trúc nào**, tùy theo nguồn lực, quy mô đội ngũ, cách tổ chức vận hành của doanh nghiệp và giai đoạn phát triển của sản phẩm.
- Biết **tư duy phân rã (decompose)** một bài toán nghiệp vụ thực tế (thương mại điện tử) thành các module độc lập theo ranh giới nghiệp vụ, để xây dựng đúng một Modular Monolithic — đây là phần khó và quan trọng nhất của buổi học.
- Hiểu vì sao Modular Monolithic là bước đệm hợp lý cho phần lớn sản phẩm, và biết chính xác cần thay đổi những gì khi một module cần tách thành microservice độc lập sau này.
- Phân biệt được lời gọi **đồng bộ** (RESTful API, gRPC) và **bất đồng bộ** (message queue với RabbitMQ) giữa các module/service, và biết khi nào nên dùng loại nào trong cùng một luồng nghiệp vụ.
- Có khả năng cài đặt RabbitMQ cơ bản, viết code publish/subscribe message giữa các module.
- Có khả năng demo một lời gọi đồng bộ bằng RESTful hoặc gRPC giữa hai module.
- Chuẩn bị tốt cho phần thuyết trình và demo.

2. Nội dung cần nghiên cứu và chuẩn bị

2.1. Monolithic vs Modular Monolithic vs Microservices — khi nào dùng cái nào

- **Monolithic (truyền thống):** Toàn bộ ứng dụng là một codebase, một process, thường dùng chung một database.
    - Ưu điểm: đơn giản để phát triển, build, test, deploy — rất phù hợp đội nhỏ, sản phẩm giai đoạn đầu.
    - Nhược điểm: codebase dễ rối dần khi lớn lên nếu không có kỷ luật tổ chức module (spaghetti code), khó scale riêng từng phần, một lỗi ở một chỗ có thể ảnh hưởng đến cả hệ thống, nhiều team cùng sửa một codebase dễ đụng nhau.
- **Microservices:** Mỗi nghiệp vụ là một service độc lập, có database riêng, deploy độc lập, scale độc lập, có thể dùng công nghệ khác nhau cho từng service.
    - Ưu điểm: scale độc lập theo đúng phần cần, các team làm việc song song không giẫm chân nhau, dễ thay thế/nâng cấp từng phần.
    - Nhược điểm: **phức tạp vận hành hơn rất nhiều** — cần hạ tầng CI/CD cho từng service (Buổi 10, 11), service discovery, giám sát/log tập trung, xử lý lỗi phân tán (một service sập không được kéo sập cả hệ thống), transaction trải dài nhiều service khó xử lý hơn hẳn một database transaction thông thường. Đòi hỏi đội ngũ DevOps đủ mạnh và nguồn lực hạ tầng đủ lớn.
- **Modular Monolithic:** Nằm giữa hai thái cực trên — vẫn là **một codebase, một process, một đơn vị deploy** (đơn giản như Monolithic), nhưng code được **tổ chức chặt chẽ thành các module độc lập theo ranh giới nghiệp vụ rõ ràng**, giao tiếp với nhau qua interface thay vì gọi chéo tùy tiện.
    - Ưu điểm: vận hành đơn giản như Monolithic (một pipeline CI/CD, một lần deploy) nhưng có kỷ luật code gần với Microservices, dễ dàng tách dần từng module ra thành service riêng khi thực sự cần, mà không phải viết lại từ đầu.
- **Tiêu chí thực tế để chọn kiến trúc — phụ thuộc vào:**
    - **Quy mô đội ngũ:** Đội nhỏ (vài người, một nhóm sinh viên) → Monolithic/Modular Monolithic dễ quản lý hơn nhiều lần. Đội lớn với nhiều team độc lập phụ trách từng mảng nghiệp vụ → Microservices giúp các team phát triển, deploy song song mà không phụ thuộc lẫn nhau.

- **Nguồn lực hạ tầng & vận hành:** Microservices đòi hỏi hạ tầng và đội ngũ DevOps đủ mạnh; một startup/công ty nhỏ thiếu nguồn lực này rất dễ "chết" vì gánh nặng vận hành trước khi kịp có được lợi ích scale.
- **Giai đoạn của sản phẩm:** Sản phẩm mới, mô hình nghiệp vụ chưa ổn định, ranh giới các nghiệp vụ còn có thể thay đổi → Modular Monolithic giúp dễ dàng tái cấu trúc ranh giới module khi hiểu nghiệp vụ rõ hơn. Sản phẩm đã trưởng thành, biết rõ phần nào cần scale riêng (ví dụ module Thanh toán cần cô lập, cần scale riêng vào đợt sale) → tách đúng phần đó thành microservice.
- **Nguyên tắc nhiều công ty thực tế áp dụng ("MonolithFirst"):** Không bắt đầu bằng Microservices ngay từ ngày đầu. Bắt đầu với Modular Monolithic, chỉ tách một module ra thành service độc lập khi có **lý do rõ ràng** (team cần tách biệt, cần scale riêng, cần công nghệ khác, cần cô lập vì bảo mật/compliance...), không tách "cho oai" hay tách toàn bộ một lượt.

2.2. Tư duy phân rã nghiệp vụ (Domain Decomposition) — ví dụ bài toán Thương mại điện tử

Đây là phần khó nhất và cũng là trọng tâm của buổi học: **làm sao để tư duy ra được ranh giới module đúng**, không phải chia module theo cảm tính.

- **Bounded Context (mượn từ Domain-Driven Design) làm nền tảng tư duy:** Một module nên bao trùm một "ngữ cảnh nghiệp vụ" trọn vẹn — có khái niệm, ngôn ngữ nghiệp vụ riêng, và hạn chế phụ thuộc vào chi tiết cài đặt bên trong của module khác.
- **Bài toán ví dụ — Hệ thống bán hàng thương mại điện tử (E-commerce):** Liệt kê các nghiệp vụ chính cần có: quản lý sản phẩm (**Catalog**), giỏ hàng (**Cart**), đặt hàng (**Order**), thanh toán (**Payment**), quản lý tồn kho (**Inventory**), vận chuyển (**Shipping**), người dùng/xác thực (**Identity**), thông báo (**Notification**), khuyến mãi (**Promotion**), đánh giá sản phẩm (**Review**).
- **Bộ câu hỏi để tư duy tách module** (áp dụng cho từng nghiệp vụ ở trên):
  1. Nghiệp vụ này có khái niệm/ngôn ngữ riêng, khác biệt rõ với các nghiệp vụ khác không? (Ví dụ: "SKU", "tồn kho khả dụng" là ngôn ngữ riêng của Inventory, khác với "giỏ hàng", "mã giảm giá" của Cart/Promotion.)
  2. Trong tổ chức thật, ai/bộ phận nào chịu trách nhiệm nghiệp vụ này? (Team Kho vận khác team Marketing, khác team Kế toán/Thanh toán — ranh giới trách nhiệm trong tổ chức thường phản ánh đúng ranh giới module nên có.)
  3. Dữ liệu của nghiệp vụ này có thể đứng độc lập, không cần `JOIN` trực tiếp xuyên suốt với dữ liệu của module khác không?
  4. Tốc độ thay đổi yêu cầu của nghiệp vụ này có khác các nghiệp vụ khác không? (Ví dụ: Promotion thay đổi liên tục theo từng chiến dịch, trong khi Catalog tương đối ổn định — tách riêng để thay đổi module này không ảnh hưởng module kia.)
- **Kết quả tách module cho ví dụ trên:** `Catalog`, `Cart`, `Order`, `Payment`, `Inventory`, `Shipping`, `Identity`, `Notification`, mỗi module gồm:
  - Một thư mục/project code riêng biệt, ranh giới rõ ràng (namespace/solution project riêng).
  - Bên trong module vẫn có thể áp dụng lại Layered/Clean Architecture đã học ở Buổi 4 (Domain, Application, Infrastructure) — nhưng phạm vi chỉ trong module đó.
  - **Quan trọng nhất: mỗi module có schema/database riêng** (có thể vẫn chung một database server để đơn giản hóa vận hành, nhưng schema/table tách biệt) — module khác **tuyệt đối không được** query thẳng vào bảng của module khác. Đây chính là điều kiện tiên quyết để sau này tách database ra riêng khi lên microservice mà không phải thiết kế lại.
- **Quy tắc giao tiếp giữa các module trong Modular Monolithic:** Module A không được gọi thẳng vào Repository/DbContext của module B. Module B phải "expose" ra một giao diện dịch vụ (ví dụ một

interface `IInventoryService`) để module A gọi qua đó. Đây chính là "đường cắt" sẽ được thay bằng lời gọi qua network (REST/gRPC/RabbitMQ) khi module B được tách thành microservice sau này — logic nghiệp vụ bên trong không cần viết lại, chỉ thay cách gọi.

2.3. Con đường tiến hóa từ Modular Monolithic lên Microservices

- Nhờ mỗi module trong Modular Monolithic đã có sẵn: schema riêng, giao tiếp qua interface (không phụ thuộc chéo trực tiếp vào chi tiết cài đặt của module khác), việc tách một module cụ thể thành service độc lập sau này chỉ cần 4 bước:
  1. Tách schema/database của module đó ra một database vật lý riêng.
  2. Thay lời gọi interface nội bộ (in-process, cùng tiến trình) bằng lời gọi qua network — đồng bộ (REST/gRPC) hoặc bất đồng bộ (RabbitMQ) tùy tình huống (xem mục 2.4–2.6).
  3. Đóng gói module đó thành một ứng dụng/image Docker riêng (kế thừa Buổi 6, 7).
  4. Có pipeline CI/CD riêng cho service mới tách ra (kế thừa Buổi 10, 11).
- **Không tách hết mọi module cùng lúc:** Trong thực tế, doanh nghiệp thường tách dần **từng module có lý do rõ ràng nhất trước** (ví dụ tách `Payment` trước vì lý do bảo mật/tuân thủ (compliance), hoặc tách `Inventory` vì cần scale riêng vào mùa sale), các module còn lại vẫn ở trong monolith cho đến khi có lý do đủ mạnh để tách tiếp. Đây gọi là **kiến trúc lai (hybrid)** — rất phổ biến trong thực tế, khác với hiểu lầm phổ biến rằng phải "100% microservices" ngay từ đầu.

2.4. Giao tiếp đồng bộ (Synchronous) giữa các Module/Service

- **RESTful API:** Gọi trực tiếp qua HTTP, bên gọi chờ phản hồi ngay rồi mới tiếp tục xử lý. Phù hợp khi **cần kết quả ngay lập tức** để quyết định bước tiếp theo — ví dụ `Order` cần hỏi `Inventory` "còn hàng không" trước khi xác nhận đơn hàng.
- **gRPC:** Giao thức RPC hiệu năng cao, dùng HTTP/2 và Protocol Buffers (dữ liệu dạng binary, gọn hơn JSON nhiều), định nghĩa contract rõ ràng qua file `.proto`. Phù hợp cho giao tiếp **nội bộ giữa các service backend với nhau** (service-to-service), nơi cần độ trễ thấp, không cần thân thiện với trình duyệt như REST.
- **So sánh nhanh REST vs gRPC:**
  - REST: dễ debug (đọc được bằng mắt qua JSON), phổ biến, phù hợp để **expose ra bên ngoài** (public API cho frontend, đối tác).
  - gRPC: nhanh hơn, contract chặt chẽ (kiểu dữ liệu được định nghĩa sẵn qua `.proto`, tránh lỗi do gõ sai field), phù hợp giao tiếp **nội bộ giữa các service** với nhau.
- **Rủi ro của gọi đồng bộ dây chuyền:** Nếu `Order` gọi đồng bộ tới `Inventory`, `Inventory` lại gọi đồng bộ tới một service khác... một service chậm hoặc sập có thể kéo sập cả chuỗi (cascading failure). Đây là lý do cần cân nhắc chuyển các bước không cần phản hồi ngay sang bất đồng bộ.

2.5. Giao tiếp bất đồng bộ (Asynchronous) với RabbitMQ

- **Vì sao cần bất đồng bộ:** Không phải mọi bước trong một luồng nghiệp vụ đều cần phản hồi ngay lập tức. Ví dụ sau khi `Order` tạo đơn hàng thành công: gửi email xác nhận (`Notification`), cập nhật số liệu thống kê, hay cập nhật lượt dùng mã giảm giá (`Promotion`) đều có thể xử lý "sau đó một chút" mà không cần `Order` phải chờ — giúp phản hồi nhanh cho người dùng và giảm phụ thuộc chặt (loose coupling) giữa các module.
- **Message Queue là gì:** Một hệ thống trung gian nhận message từ bên gửi (Producer/Publisher) và lưu tạm, để bên nhận (Consumer/Subscriber) lấy ra xử lý theo tốc độ của riêng mình — Producer và Consumer không cần hoạt động cùng lúc, không cần biết sự tồn tại của nhau.

- **Các khái niệm cơ bản của RabbitMQ cần tìm hiểu:**
  - **Exchange:** Nơi nhận message từ Producer, quyết định message sẽ được chuyển tới (các) Queue nào, dựa theo loại exchange (`direct`, `topic`, `fanout`) và routing key.
  - **Queue:** Nơi lưu trữ message, chờ Consumer lấy ra xử lý.
  - **Binding:** Liên kết giữa một Exchange và một Queue.
  - **Producer / Consumer:** Bên gửi và bên nhận message.
  - **Publish/Subscribe với `fanout` exchange:** Một sự kiện (ví dụ `OrderCreated`) được gửi tới **nhiều Queue khác nhau cùng lúc**, mỗi Queue ứng với một module quan tâm đến sự kiện đó — mỗi module xử lý độc lập, module này không biết và không cần quan tâm module kia xử lý ra sao.
- **Ví dụ áp dụng vào bài toán E-commerce:** Khi `Order` tạo đơn hàng thành công, publish một event `OrderCreated` lên RabbitMQ. Module `Inventory` subscribe event này để trừ tồn kho, module `Notification` subscribe để gửi email xác nhận, module `Promotion` subscribe để cập nhật lượt dùng mã giảm giá — cả ba xử lý song song, độc lập; `Order` publish xong là coi như hoàn tất phần việc của mình, không chờ các module kia xử lý xong.

2.6. Chọn đồng bộ hay bất đồng bộ — nguyên tắc quyết định

- **Cần kết quả ngay để quyết định bước tiếp theo** (ví dụ phải biết còn hàng hay không trước khi cho phép đặt hàng) → dùng **đồng bộ** (REST/gRPC).
- **Không cần phản hồi ngay, chỉ cần báo cho module khác biết "có việc gì đó vừa xảy ra" để tự xử lý theo nhịp riêng** → dùng **bất đồng bộ** (RabbitMQ).
- Một luồng nghiệp vụ thực tế thường **kết hợp cả hai**: ví dụ `Order` gọi đồng bộ (REST/gRPC) tới `Inventory` để kiểm tra tồn kho trước khi tạo đơn, nhưng ngay sau khi tạo đơn thành công thì publish event bất đồng bộ để các module còn lại tự xử lý phần việc của mình.

3. Chuẩn bị thuyết trình và demo

- **Slide:** Giải thích Monolithic vs Modular Monolithic vs Microservices và tiêu chí lựa chọn theo nguồn lực/tổ chức (mục 2.1); trình bày rõ tư duy phân rã domain bằng ví dụ E-commerce (mục 2.2) — đây nên là phần được đầu tư kỹ nhất trong slide; sơ đồ các module và các đường giao tiếp đồng bộ/bất đồng bộ giữa chúng.
- **Demo — bắt buộc gồm 3 phần, dựa trên cùng một project Modular Monolithic cho bài toán E-commerce:**
  1. **Cấu trúc Modular Monolithic:** Xây dựng (hoặc trình bày cấu trúc code đã chuẩn bị sẵn) với ít nhất 3 module, ví dụ `Order`, `Inventory`, `Notification`. Mỗi module có schema database riêng, giao tiếp nội bộ qua interface, **không** truy cập chéo trực tiếp vào DbContext/Repository của module khác. Trình bày rõ cấu trúc thư mục thể hiện ranh giới từng module.
  2. **Bắt buộc — demo lời gọi đồng bộ:** Module `Order` gọi module `Inventory` để kiểm tra tồn kho bằng RESTful API hoặc gRPC (chọn ít nhất một, khuyến khích thử cả hai nếu còn thời gian) trước khi xác nhận đơn hàng. Demo cả hai tình huống: đủ hàng (tạo đơn thành công) và hết hàng (từ chối tạo đơn).
  3. **Bắt buộc — demo lời gọi bất đồng bộ với RabbitMQ:** Sau khi `Order` tạo đơn thành công, publish event `OrderCreated` lên RabbitMQ; có ít nhất 2 Consumer khác nhau (ví dụ `Inventory` trừ kho, `Notification` in log/gửi email giả lập) cùng nhận và xử lý event độc lập. Chứng minh tính bất đồng bộ bằng cách tắt một Consumer đi trước khi tạo đơn — Producer vẫn publish message bình thường, message vẫn nằm chờ trong queue cho đến khi Consumer đó chạy lại và xử lý.

4. **Giải thích trước lớp (không cần code thật):** Nếu sau này cần tách `Inventory` thành một microservice độc lập, chỉ ra rõ những phần nào trong code hiện tại cần thay đổi (thay lời gọi interface nội bộ bằng lời gọi qua network, tách schema thành database riêng, đóng gói thành image riêng, có pipeline CI/CD riêng) — chứng minh nhờ tư duy module hóa từ đầu nên việc tách không cần viết lại toàn bộ logic nghiệp vụ.

- **Thời gian:** Ưu tiên thời lượng nhiều nhất cho phần tư duy phân rã domain (mục 2.2, đây là phần khó và quan trọng nhất) và phần demo bất đồng bộ với RabbitMQ.

4. Phân công công việc và ước lượng thời gian (Gợi ý)

Task	**Người thực hiện (Gợi ý)**	**Ước lượng (Point)**
Nghiên cứu Monolithic/Modular Monolithic/Microservices và tiêu chí lựa chọn	TV1	3
Nghiên cứu Bounded Context, tư duy phân rã domain (DDD ở mức cơ bản)	TV2	3
Thiết kế bài toán ví dụ E-commerce: liệt kê nghiệp vụ, xác định ranh giới module	Cả nhóm	4
Nghiên cứu RESTful vs gRPC cho giao tiếp đồng bộ	TV1	3
Nghiên cứu RabbitMQ (Exchange, Queue, Binding, Pub/Sub)	TV2	3
Chuẩn bị nội dung slide thuyết trình	Nhóm trưởng	2
Dựng project Modular Monolithic demo (3 module, schema riêng, giao tiếp qua interface)	TV3, TV1	6
Cài đặt RabbitMQ và viết Producer/Consumer demo	TV2, TV3	5
Viết demo gọi đồng bộ REST/gRPC giữa 2 module	TV1	4
Tổng hợp, kiểm tra và chạy thử demo	Cả nhóm	2
Luyện tập thuyết trình và demo	Cả nhóm	1
Tổng cộng		**36**

5. Tài liệu tham khảo

- **Tài liệu chính thức:** RabbitMQ Documentation (rabbitmq.com/docs), gRPC Documentation (grpc.io/docs), Protocol Buffers Documentation.
- **Bài viết:** "MonolithFirst" của Martin Fowler; các bài viết về Modular Monolith (ví dụ kinh nghiệm thực tế của Shopify khi vận hành modular monolith ở quy mô lớn); tài liệu nhập môn về Domain-Driven Design và Bounded Context.
- **Video:** Các video so sánh Monolithic/Modular Monolithic/Microservices, hướng dẫn RabbitMQ Pub/Sub cơ bản, hướng dẫn xây dựng gRPC service.

6. Ghi chú quan trọng

- **Không có kiến trúc nào "luôn đúng":** Microservices không phải lúc nào cũng tốt hơn Monolithic — lựa chọn sai kiến trúc so với nguồn lực và giai đoạn thực tế của đội ngũ/sản phẩm thường gây hại nhiều hơn lợi. Khi thuyết trình, tránh trình bày Microservices như một "đích đến bắt buộc".
- **Ranh giới module là phần khó nhất, không có đáp án tuyệt đối duy nhất:** Việc chia module có thể có nhiều cách hợp lý khác nhau tùy góc nhìn — quan trọng là **giải thích được lý do** tại sao chia như vậy, dựa trên bộ câu hỏi ở mục 2.2, chứ không phải học thuộc một cách chia cố định.
- **Đừng để module giao tiếp chéo trực tiếp vào dữ liệu của nhau:** Đây là lỗi phổ biến nhất khiến một "Modular Monolithic" trên danh nghĩa thực chất vẫn là một Monolithic rối rắm thông thường, không thể tách ra sau này.
- **Bất đồng bộ không có nghĩa là "không đáng tin cậy":** RabbitMQ đảm bảo message được lưu trữ và xử lý ngay cả khi Consumer tạm thời không hoạt động — cần làm rõ điều này khi demo để tránh hiểu lầm rằng bất đồng bộ đồng nghĩa với dễ mất dữ liệu.
- **Đây là buổi tổng hợp kiến thức của cả môn học:** Modular Monolithic ở buổi này có thể tận dụng lại toàn bộ kiến thức từ Clean/Layered Architecture (Buổi 4), Docker (Buổi 6, 7), CI/CD nhiều môi trường (Buổi 10, 11) khi mô phỏng việc tách một module ra microservice thật sự.

Chúc các bạn học tập hiệu quả và thành công!